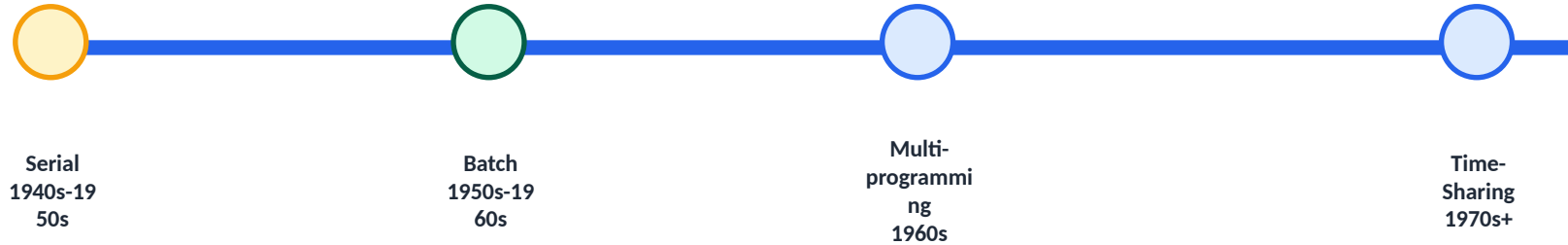


Operating System

Processing Models & Execution Modes

Serial • Batch • Multiprogramming • Time-Sharing

Evolution of OS Processing Models



Key Characteristics:

Model	Resource Type	CPU Util.	I/O Util.	User Interaction
Serial	Single User	Low	Low	None
Batch	Multiple Jobs	Medium	Medium	None
Multiprogramming	Multiple Jobs	High	High	None
Time-Sharing	Multiple Users	Very High	Very High	Yes

Serial Processing (Late 1940s - Mid 1950s)

Historical Context:

- No operating systems existed during this period
- Programmers interacted directly with hardware
- Programs written in machine language
- Input via card readers, output via line printers
- Display lights indicated error conditions

How Serial Processing Worked:

- Programmers reserved machine time on a sign-up sheet
- Users accessed computer sequentially (one after another)
- After time block finished, next programmer's turn
- Output appeared after minutes, hours, or even days
- Result: Wasted computer time and low user productivity

SERIAL PROCESSING

(Late 1940s – Mid-1950s)



No operating systems existed.

101010
010101
101010

Programmers interacted directly with hardware using machine language.



Programs and data were entered through punched cards.



Users reserved computer time and used the machine one after another (serially).



System software such as compilers, loaders, linkers, and debuggers were later developed to simplify programming.



HOW SERIAL PROCESSING WORKED

1. PROGRAMMER



Writes program in machine language

2. PUNCHED CARDS



Program and data punched on cards

3. SUBMIT & WAIT



User submits job and waits for turn

4. COMPUTER EXECUTION



Job runs to completion on the machine

5. OUTPUT



Results are printed and returned

SYSTEM SOFTWARE (developed later)



COMPILER
Translates source code to machine language



LOADER
Loads program into memory



LINKER
Combines modules and libraries



DEBUGGER
Helps find and fix errors

LIMITATIONS



Low CPU and resource utilization.



Significant idle time when users finished early.



Users had to wait for their turn.

Serial Processing: Problems & Improvements

Main Problem

Wasted Computer Time

Programmers might book 2 hours but finish in 1.5 hours.
No other programmer could use the computer during unused time.



System Software Tools

- Language translators
- Loaders & Linkers
- Debuggers
- Library routines
- I/O routines

Impact of System Software Tools:

- Language Translators (Compilers/Interpreters) - Converted programs from high-level languages to executable code
- Loaders - Automated the process of loading executable programs into memory
- Debuggers - Detected and examined run-time errors
- Linkers - Linked precompiled routines with object code for execution
- **Result** - Made serial processing more efficient, but still had low resource utilization

Batch Processing (Mid 1950s - 1960s)

Historical Context:

- Introduction of transistors made computers more reliable- expectation that they would continue to work for a long time
- **Mainframes** generally operated by the professional operators
- Computers became expensive and could be afforded only by major corporations and government agencies
- First operating system developed by General Motors for IBM 701
- Clear separation made between computer operators and programmers

Key Concept:

Jobs with similar requirements are batched together and run one after another

Example: All FORTRAN jobs run together so the compiler is loaded only once

To improve resource utilization and user productivity, **the first operating system was developed by General Motors for IBM 701 in the mid-1950s**. This operating system was termed as a **batch operating system**.

BATCH PROCESSING

(Mid-1950s)



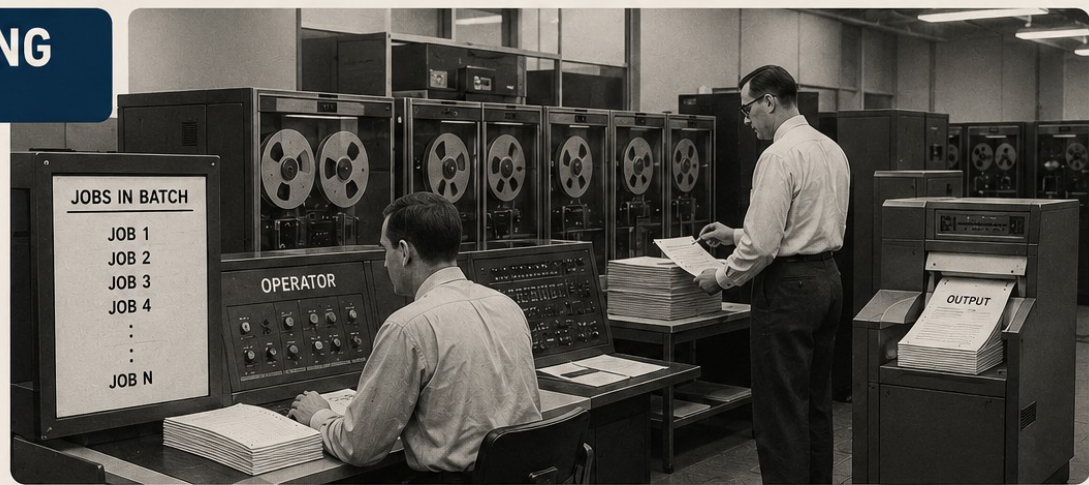
Introduction of transistors made computers more reliable.



Programmers submitted jobs (programs, data, and control information) to computer operators.



Similar jobs were grouped into batches and executed automatically.



FEATURES



First batch operating systems automatically transferred control from one job to the next.



Reduced operator intervention.



Improved resource utilization.

HOW BATCH PROCESSING WORKED

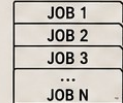
1. JOB SUBMISSION



Programmers submit jobs to operators



2. BATCH FORMATION



Similar jobs are grouped into a batch



3. BATCH EXECUTION



Operating system executes jobs automatically one after another



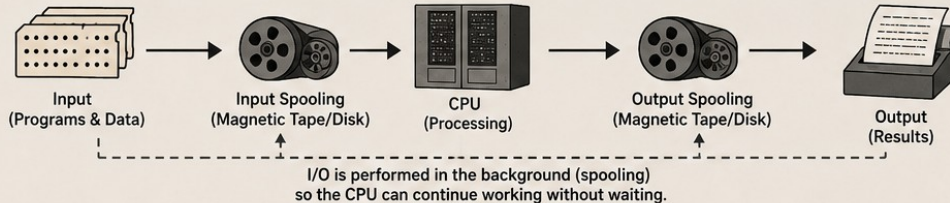
4. OUTPUT



Results are printed or stored

IMPROVEMENTS – SPOOLING

SPOOLing used magnetic tapes and later disks to reduce CPU idle time caused by slow input/output devices.



LIMITATIONS



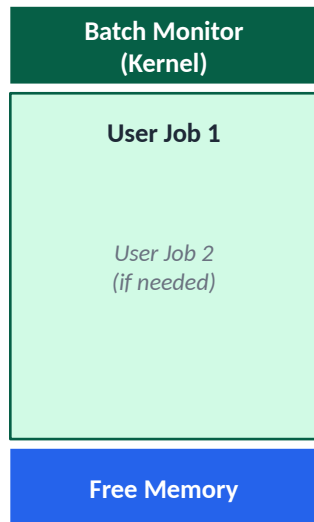
CPU still remained idle during I/O operations.



No interaction between users and running programs.

Batch Processing: Memory Layout & Operation

Memory Layout



Batch Monitor Operation:

- Reads from card reader and loads first job
- JOB_START identifies job beginning
- Executes the job
- JOB_END identifies job completion
- Searches for next job in batch
- Repeats until all jobs finished
- Waits for next batch from operator
- Operator intervention only at batch boundaries

✓ **Advantage: Automatic job transitions without operator intervention**

Batch Processing: SPOOLing Solution

Problem: Speed Mismatch

CPU Speed $\gg$ I/O Device Speed

CPU much faster than card reader/printer

Result:

- CPU idle waiting for I/O
- Poor resource utilization



Solution: SPOOLing

Simultaneous Peripheral Operation Online

- Copy jobs from cards $\rightarrow$ magnetic tape
- OS reads from faster tape (not cards)
- Output goes to tape $\rightarrow$ printer later
- Reduces CPU idle time
- Later: Tapes $\rightarrow$ Disks (even faster)
- Allows direct access to jobs

Multiprogramming (1960s - 1970s)

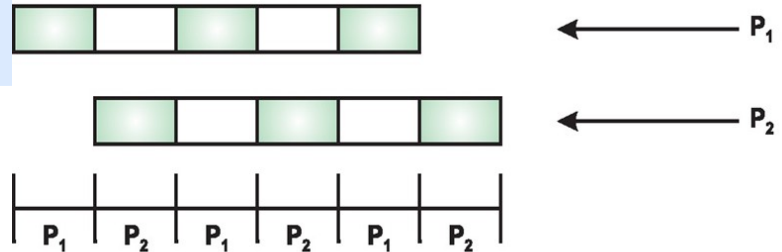
Core Idea:

Allow multiple jobs in memory at same time

When one job does I/O, CPU executes another job



(a) Serial execution of P₁ and P₂



(b) Multiprogrammed execution of P₁ and P₂

Problem & Solution:

Batch Problem:

Job P₁ needs CPU then I/O alternately. CPU idle during P₁'s I/O even if P₂ ready to run.

Multiprogramming Solution:

Multiple jobs in memory. If P₁ does I/O, execute P₂. Always have work for CPU.

✓ Result: CPU never idle as long as one job ready to execute

3. MULTIPROGRAMMING



Multiple jobs are loaded into memory simultaneously.



When one job waits for I/O, the CPU executes another job.



This reduces CPU idle time and increases system efficiency.

ADVANTAGES



Higher CPU utilization.

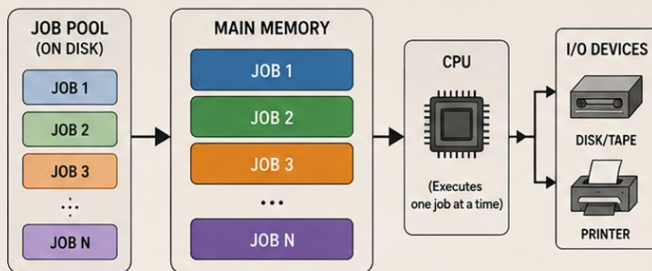


Better resource utilization.

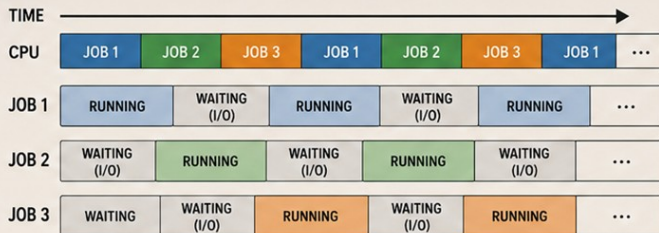


Increased throughput.

HOW MULTIPROGRAMMING WORKS



EXAMPLE EXECUTION TIMELINE



When a job waits for I/O, the CPU switches to another job.

OPERATING SYSTEM RESPONSIBILITIES



Job Scheduling:
Selecting jobs to load into memory.



Memory Management:
Managing multiple jobs in memory.



CPU Scheduling:
Choosing which job gets CPU time.

DEGREE OF MULTIPROGRAMMING



Number of jobs competing for resources at a given time.



Higher degree generally leads to better resource utilization.

EXAMPLE



LIMITATION



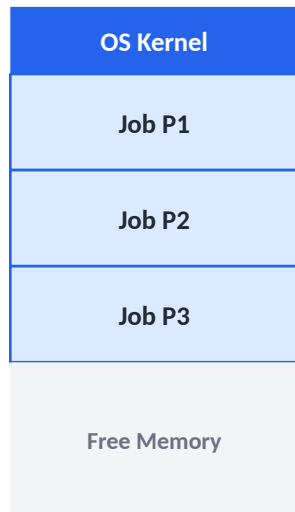
Users still had to wait a long time for output.



No direct interaction with programs.

Multiprogramming: Memory Layout & Execution

Memory Layout



Key Concepts:

- Degree of Multiprogramming - Number of jobs competing for resources
- Higher degree → Better resource utilization
- Job Scheduling - OS selects which job to load to memory
- CPU Scheduling - OS chooses which ready job gets CPU
- Memory Management - Divide memory into partitions
- Concurrent Execution - Multiple processes execute interleaved

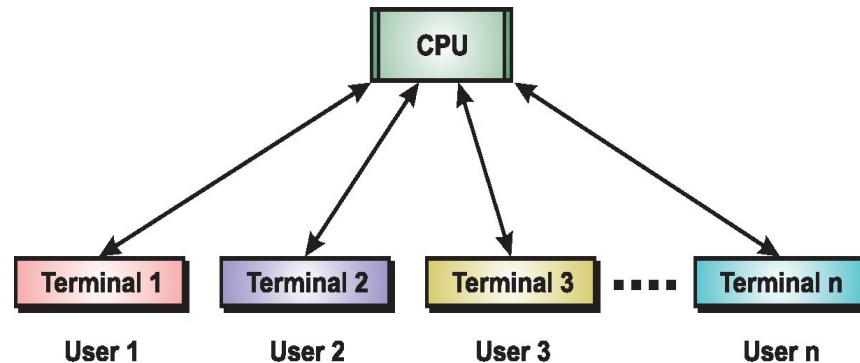
Drawbacks:

- programmers have to wait for several hours to get their output.
- Do not allow programmers to interact with the system.

Time-Sharing (Multitasking) Systems

What is Time-Sharing?

- Extension of multiprogramming systems
- Multiple users interact with system simultaneously via terminals
- Each user assigned fixed time-slot
- Users give instructions via keyboard/mouse
- System response time ~1 second
- CPU switches rapidly between users
- Each user feels they have dedicated system



Key Advantage of Time-Sharing:

Interactive Programming Environment with quick feedback (unlike batch processing's long turnaround)

Comparison of Processing Models

Aspect	Serial	Batch	Multiprogramming	Time-Sharing
Time Period	1940s-50s	1950s-60s	1960s-70s	1970s+
Users/Jobs	1 user	Multiple jobs	Multiple jobs	Multiple users
Interaction	None	None	None	Interactive
CPU Util.	Very Low	Medium	High	Very High
Response	Hours/Days	Minutes/Hours	Minutes	~1 second
Main Goal	Simplicity	Efficiency	Max CPU use	User convenience

Evolution Pattern: Serial → Batch → Multiprogramming → Time-Sharing

Each model improved resource utilization, user interaction, and system responsiveness. Modern systems use time-sharing with advanced CPU scheduling, memory management, and security.

Key Takeaways

- Serial Processing: Direct hardware access, massive resource waste
- Batch Processing: Grouped jobs, automatic transitions, SPOOLing for faster I/O
- Multiprogramming: Multiple jobs in memory, concurrent execution, CPU never idle
- Time-Sharing: Interactive multiple users, rapid CPU switching, ~1 second response
- Modern OS: Time-sharing foundation with advanced scheduling, memory, and security mechanisms